Forest Cover Type Classification

Predicting the dominant tree species of a 30 × 30 m cell from terrain and soil

Amanbaev Marlen (AAI-2601) · Erik [surname] ([group]) · Rashad [surname] ([group])

Programming for AI · Final project · Topic 37 · November 2026

581,012 cells, 7 classes, 103× imbalance

581,012

30 × 30 m cells, UCI Covertype

54

features: 10 numeric, 4 areas, 40 soils

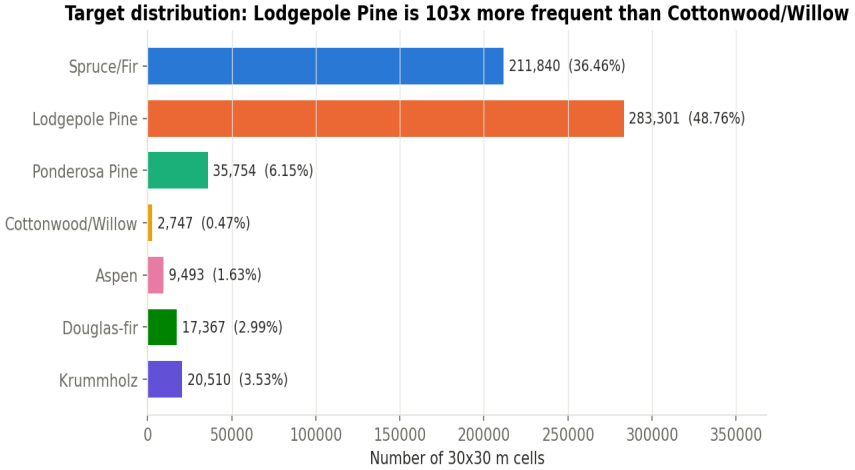
48.76%

Lodgepole Pine, largest class

0.47%

Cottonwood/Willow, smallest class

Metric: F1-macro, so that every class counts the same, not only the two big ones.



Roosevelt National Forest, Colorado: 4 wilderness areas, labels from US Forest Service inventory, features from USGS terrain data.

One pipeline, twelve modules

1 Data

`data_loader.py`

download, chunks, dtypes, Parquet

2 Audit

`audit.py`

missing, duplicates, ranges, IQR

3 EDA

`eda.py`

8 figures + interactive Plotly

4 SQL

`database.py`
`queries.py`

normalised schema, 12 queries

5 Models

`features.py`
`models.py`

Pipelines, CV tuning,
5 approaches

6 Analysis

`evaluate.py`

importance, errors, learning curve

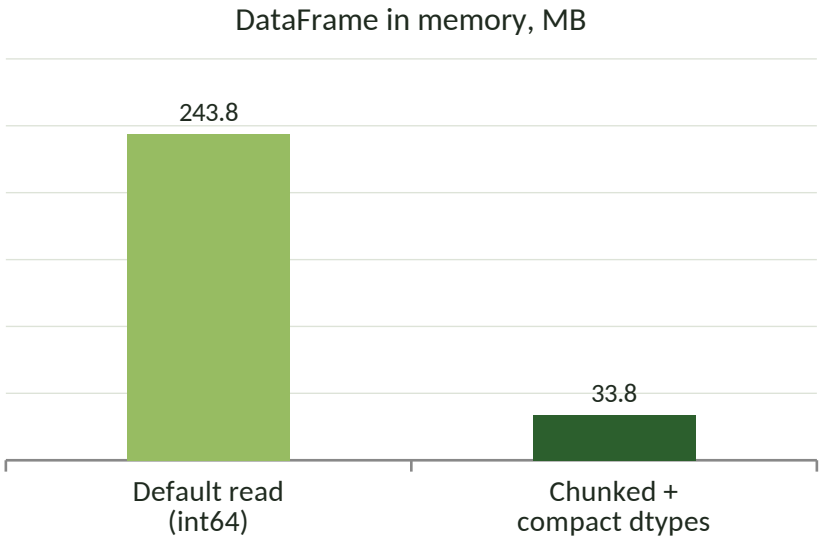
User interface

main.py: console menu with 9 operations
app.py: Streamlit dashboard with filters, SQL, models and a prediction form

Reproducible

python main.py --all runs everything
config.py holds every path & seed; 6 pytest tests; results saved as Parquet, CSV, JSON, SQLite

Compact dtypes cut memory by 86%



6.91 MB

Parquet on disk (CSV: 71.69 MB)

0.152 s

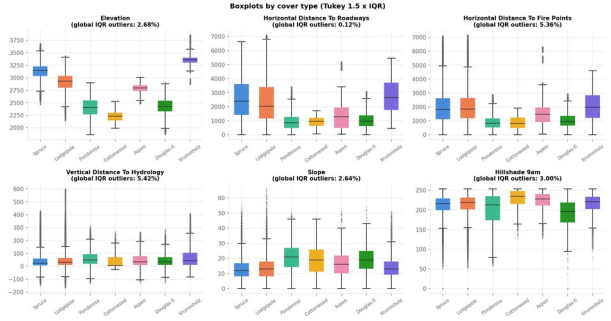
Parquet load time (CSV: 0.803 s)

- Read in chunks of 100,000 rows with int16 / uint8 dtypes
- 44 one-hot columns collapsed into 2 categorical columns
- Parquet reads 3 columns in 7 ms (columnar)

Source: reports/memory_report.json, format_benchmark.json

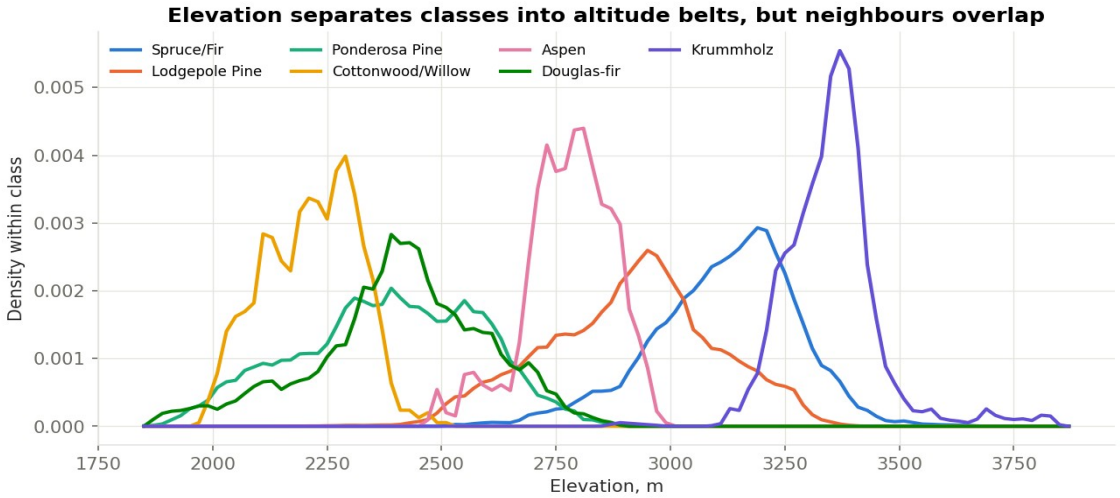
The audit found clean data, so nothing was dropped

Check	Result	Decision
Missing values	0	imputers only guard user input
Duplicate rows / conflicting labels	0 / 0	no label noise
Rows with 0 or > 1 soil flag	0	safe to collapse one-hot
Values outside physical range	0	none
IQR outliers (max column)	5.42%	kept: real terrain
Hillshade_3pm = 0	1338	kept: afternoon shadow



IQR outliers are the extremes of real terrain (remote cells, steep slopes). Dropping them would remove exactly the rare high-altitude classes.

Each species lives in its own altitude belt



Below 2,600 m: Ponderosa, Douglas-fir, Cottonwood/Willow

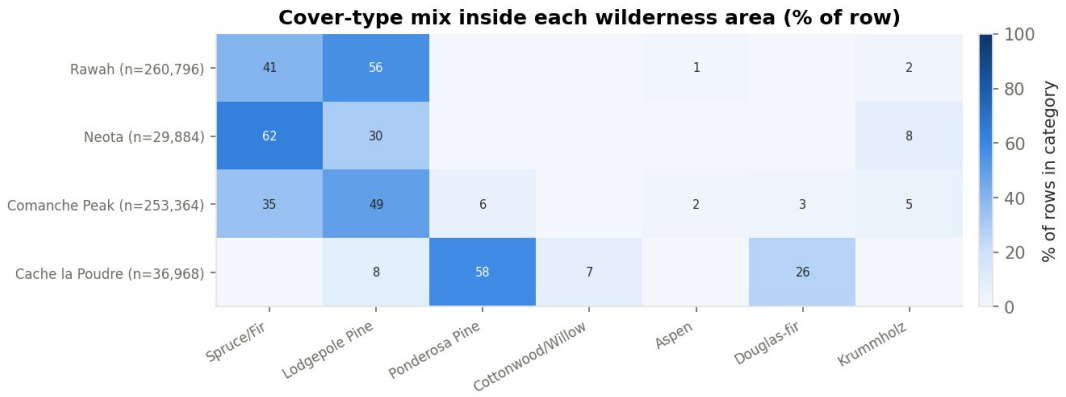
2,600–3,300 m: Lodgepole Pine, Aspen, Spruce/Fir

Above 3,300 m: Krummholz at the tree line

Belts overlap: elevation alone cannot separate Spruce/Fir from Lodgepole

Strongest correlation: Hillshade_9am ~ Hillshade_3pm, r = −0.78 (both depend on slope orientation)

Area and soil act as strong priors



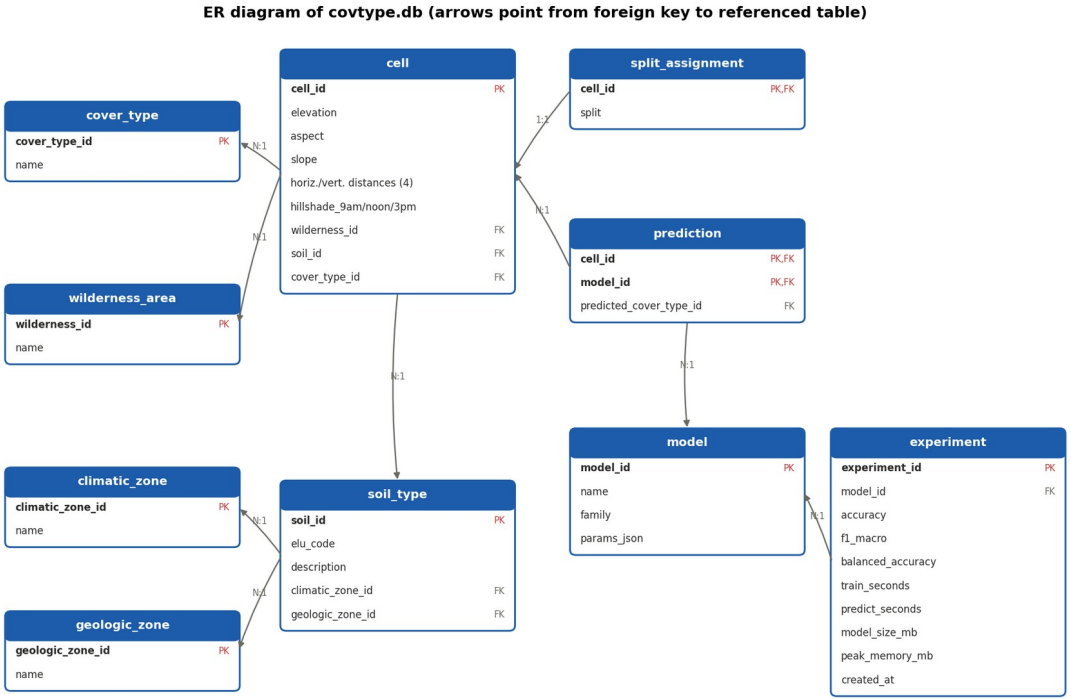
Rows: wilderness area, columns: cover type, cells: % of the area. Interactive version: 09_interactive_scatter.html

91%

of soil 12 (29,971 cells) is Lodgepole Pine (SQL Q03)

- Cottonwood/Willow grows only in Cache la Poudre
- Neota is 62% Spruce/Fir
- Ponderosa + Douglas-fir: 84% of Cache la Poudre

Normalised SQLite schema with 12 queries



JOIN up to 4 tables

GROUP BY / HAVING in 9 queries

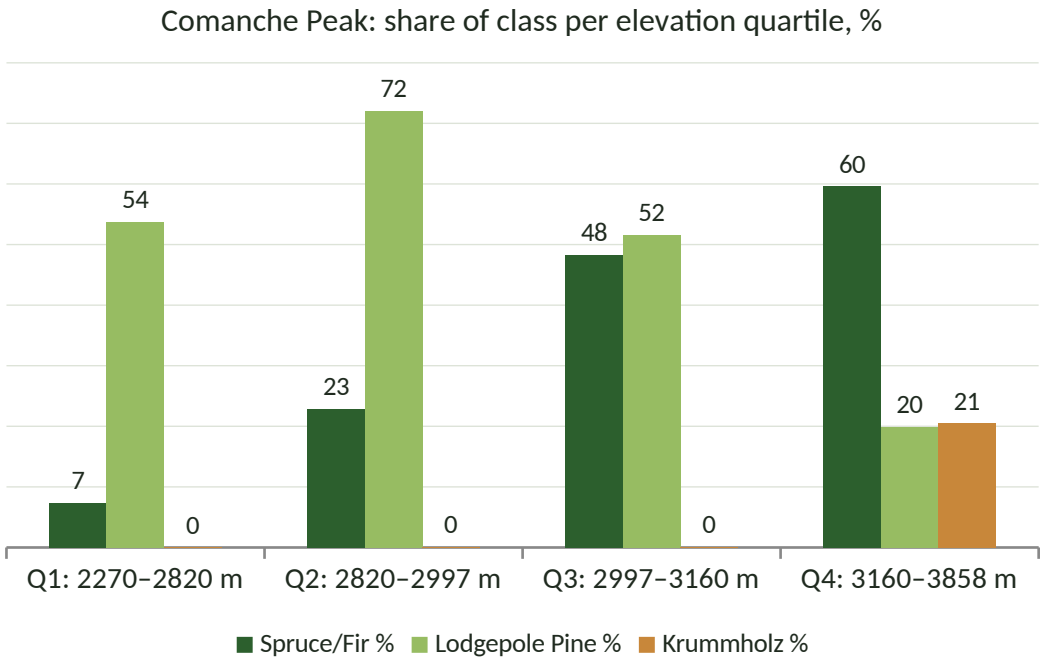
CTE and subqueries in 8 queries

Window functions: SUM OVER, ROW_NUMBER, RANK, NTILE, LAG

Model errors in SQL: predictions stored next to the data (Q10–Q12)

pandas vs SQL on the same aggregation: identical results, 0.088 s vs 0.432 s

SQL shows the altitude succession inside an area



Q08

```
NTILE(4) OVER (
  PARTITION BY wilderness_id
  ORDER BY elevation)
```

Lodgepole dominates the low quartiles, Spruce/Fir the upper ones, and Krummholz appears only in the top quartile.

Leakage-free protocol

1

Split first

stratified 80 / 20
464,809 train · 116,203 test

2

Pipelines

features + imputing + scaling
fitted on train only

3

Tune

RandomizedSearchCV, 3-fold
F1-macro, 40k-row subsample

4

Refit

best params on the full train split

5

Test once

quality, time, RAM, model size

Baseline: DummyClassifier (always the majority class)

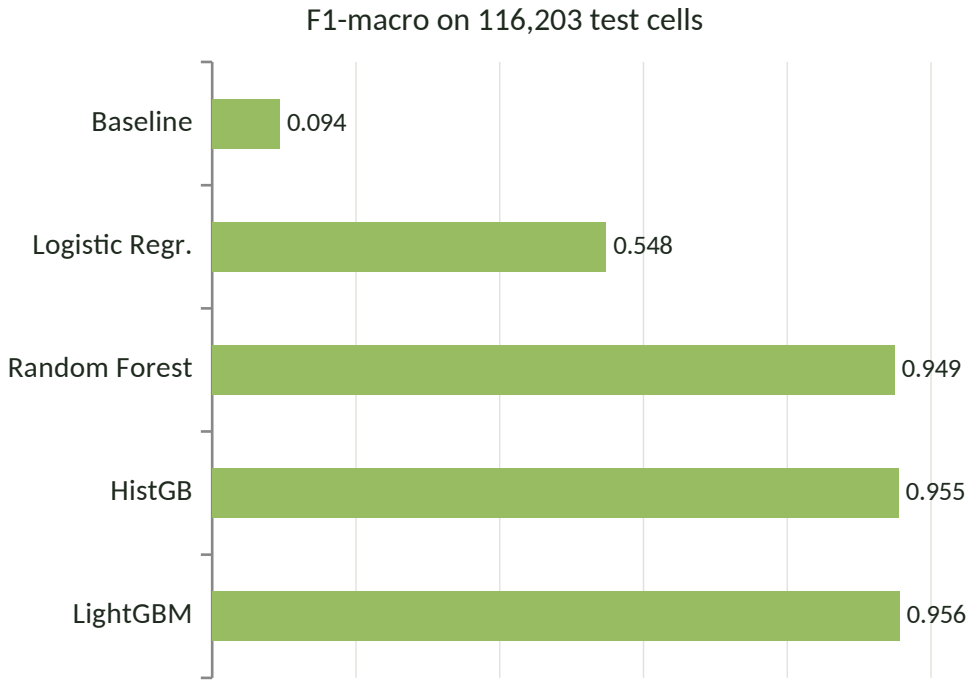
Linear: multinomial Logistic Regression, scaled + one-hot features

Bagging: Random Forest

Boosting: HistGradientBoosting and LightGBM

Imbalance: class_weight ∈ {None, balanced} searched, plus a controlled LightGBM check

Boosting wins: F1-macro 0.956



0.974

accuracy of LightGBM (baseline 0.488)

0.41

F1-macro gain of trees over the linear model

Trees model the interactions of elevation, area and soil. A linear function cannot.

Quality is close; cost decides

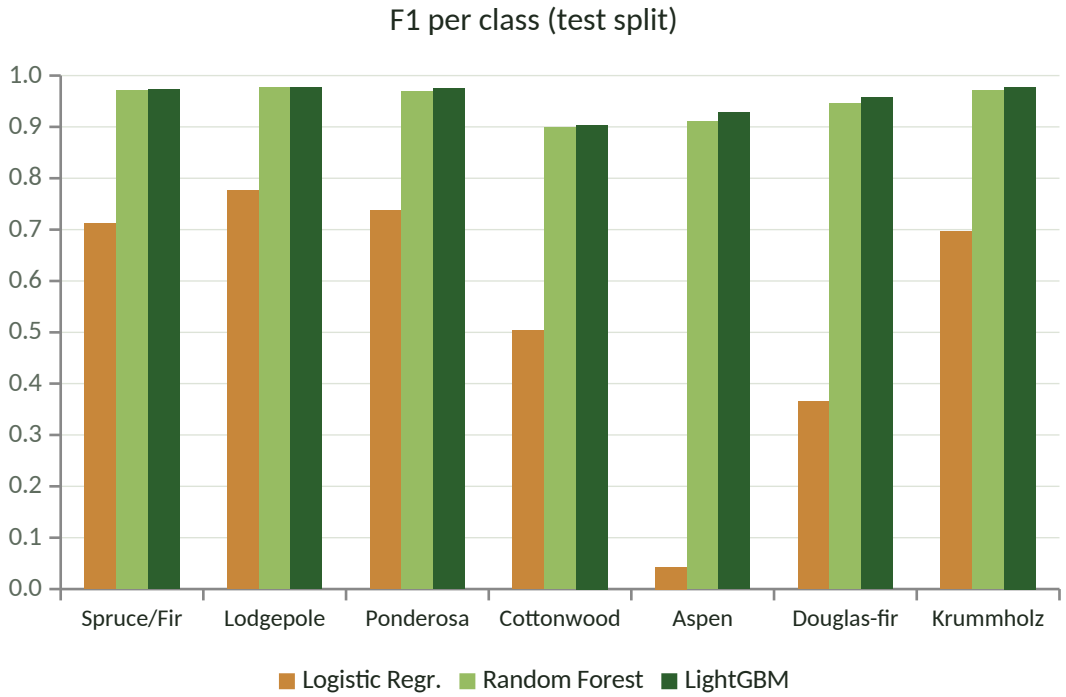
Model	F1-macro	Accuracy	Train, s	Predict ms/1k	Size, MB
Logistic Regr.	0.548	0.731	43	1	0.01
Random Forest	0.949	0.972	62	10	568
HistGB	0.955	0.970	328	101	60
LightGBM	**0.956**	**0.974**	**172**	**402**	**91**

LightGBM: best F1-macro, 91 MB (40 MB on disk), our choice

Random Forest: 3× faster to train, 39× faster to predict, but 568 MB

HistGB: close second with the smallest tree model, 2× slower to train

Rare classes separate the models



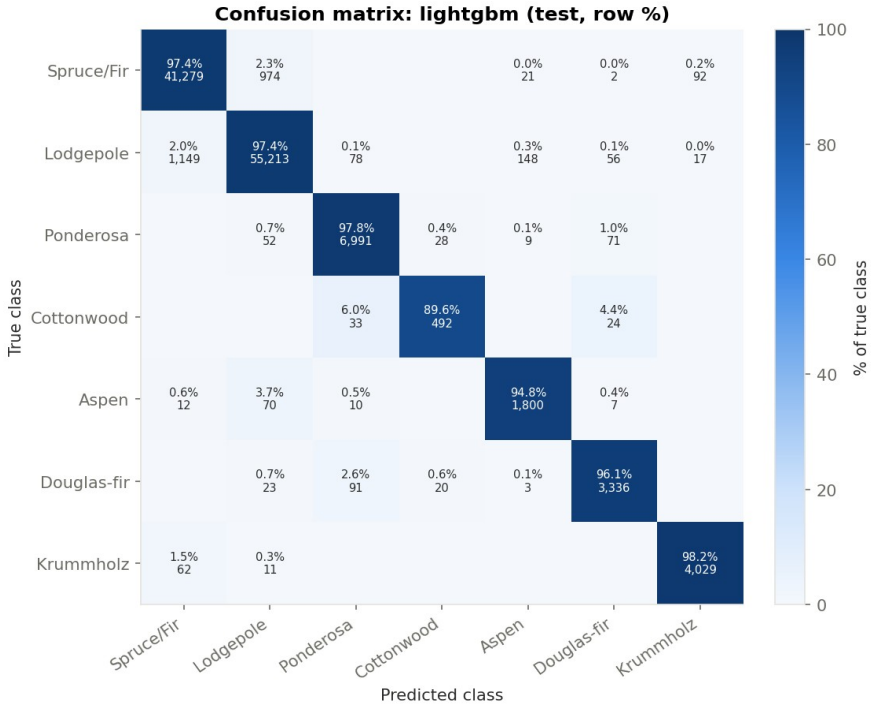
Class weights help a little

Balanced accuracy 0.947 → 0.959
Cottonwood F1 0.889 → 0.904
Accuracy unchanged (0.974)

A lesson from tuning

HistGB with lr = 0.2 and balanced weights: CV 0.8421, but F1 0.3827 after refit on all data. Capped lr at 0.1 → 0.955.

Errors happen between neighbouring belts



69%

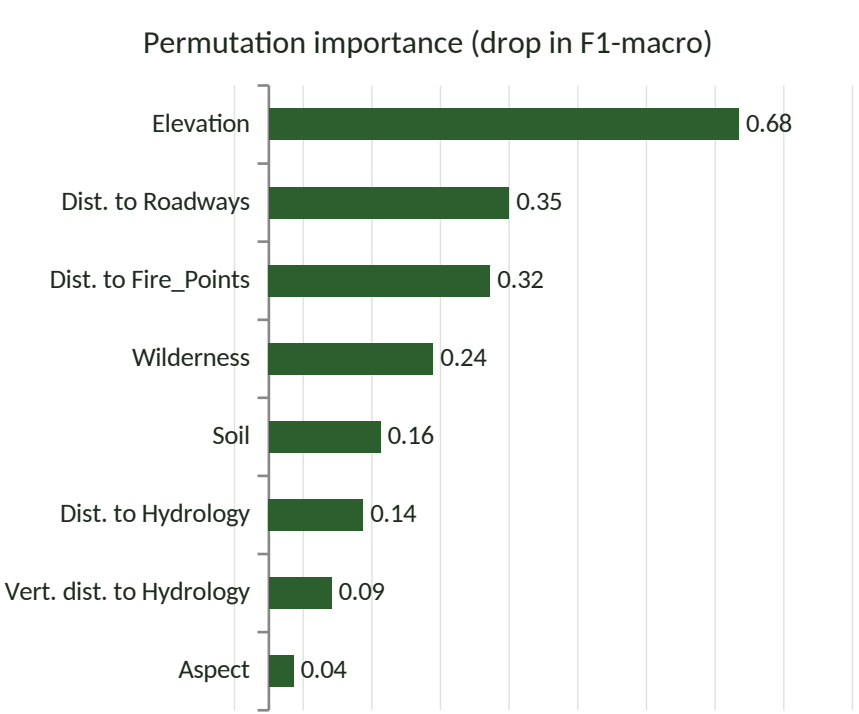
of 3,063 errors are Spruce/Fir ↔ Lodgepole Pine (SQL Q11)

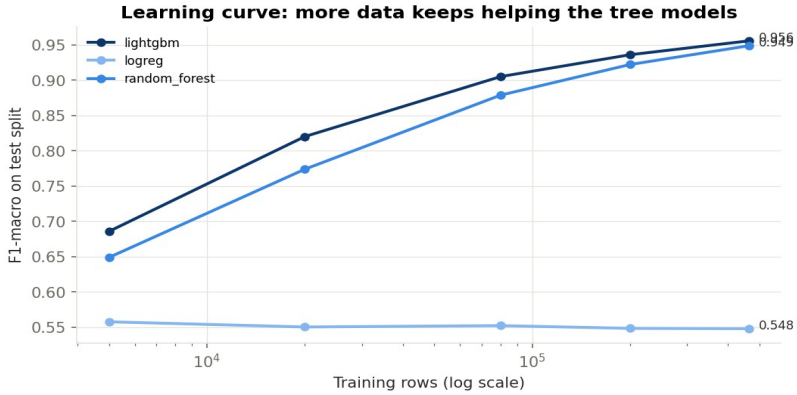
45% vs 1.9%

error rate when the top probability is < 0.6 (1.7% of cells) vs ≥ 0.6

72% of errors lie at 2,750–3,250 m. Low-confidence cells could be sent for field checks.

Elevation matters most; more data still helps



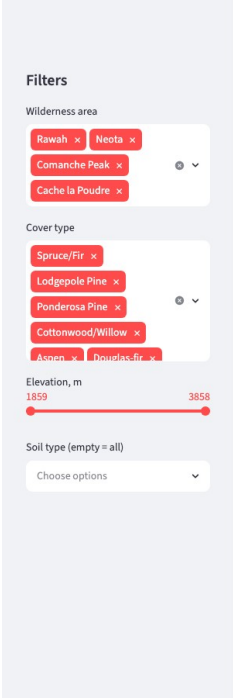


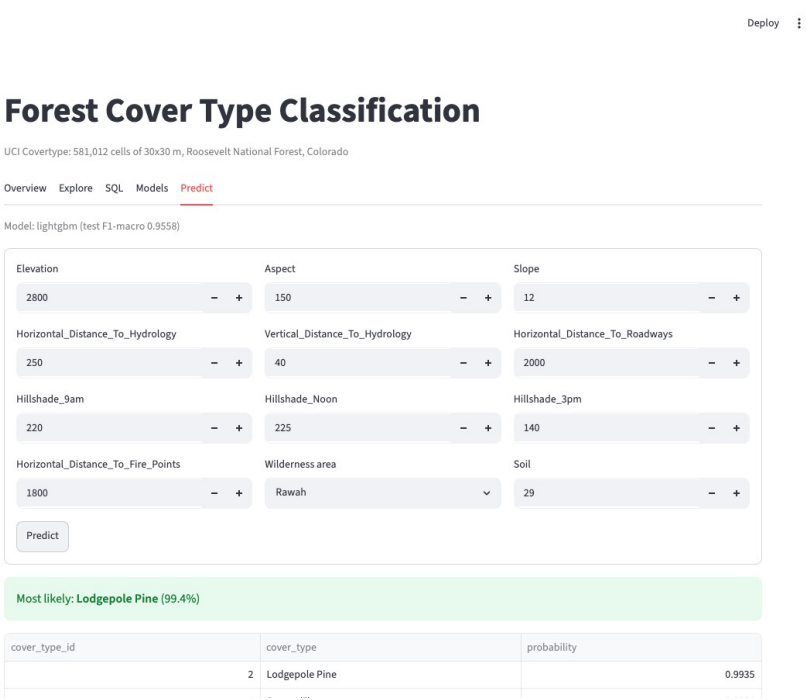
Logistic regression is flat (0.56 → 0.55): its error is bias. Trees keep improving up to all 465k rows, so the full data set is justified.

Demo: console menu and Streamlit

```
1 Prepare data
2 Data quality audit
3 Build SQLite database
4 EDA figures
5 Train and compare models
6 Interpret best model
7 Run an SQL query
8 Predict cover type
9 Show results
0 Exit
```

```
python main.py
streamlit run app.py
```





Conclusions

- Terrain + soil predict the forest type with F1-macro 0.956, accuracy 0.974
- Tree ensembles beat the linear model by 0.41 F1-macro: the classes are defined by feature interactions
- LightGBM with class weights is the best trade-off of quality, size and time
- Errors fall between adjacent vegetation belts and are flagged by low confidence
- Limitation: random split ignores spatial autocorrelation, so the score is optimistic for new regions

Team contributions

Amanbaev Marlen AAI-2601	Data engineering: chunked loading, dtypes, Parquet; SQLite schema, ER diagram, 12 SQL queries; console menu, Streamlit app, README, tests
Erik [surname] [group]	Data quality audit and cleaning decisions; exploratory analysis and all EDA figures; interactive Plotly chart; report sections 2–5
Rashad [surname] [group]	Feature engineering and pipelines; CV tuning of 5 approaches; evaluation, permutation importance, learning curve, error analysis; report sections 6–9

Report and slides written together; each member can explain every module.